

Практическое занятие Построение логической модели АСОИУ.

Часть 1. Диаграмма автоматов.

Цель занятия: получить навыки построения диаграмм автоматов UML.

Теоретические сведения к практическому занятию

Диаграмма автоматов обычно используется для описания поведения объекта в зависимости от состояния. Объект по-разному реагирует на одно и то же событие в зависимости от того, в каком состоянии он находится. Диаграммы конечного автомата обычно применяются к объектам, но могут применяться к любому элементу, который имеет поведение по отношению к другим объектам, таким как: субъекты, варианты использования, методы, подсистемы, системы и т. д.

Диаграмма автоматов - это графическое изображение состояний, которые бывают простыми и составными, и переходов, соединяющих состояния.

Рамбо определяет состояние: «Состояние - это абстракция значений атрибутов и связей объекта. Наборы значений сгруппированы вместе в состояние в соответствии со свойствами, которые влияют на общее поведение объекта».

Каждая диаграмма автоматов обычно начинается с темного круга, который указывает начальное состояние, и заканчивается кружком с рамкой, который обозначает конечное состояние.

Состояния представлены прямоугольниками со скругленными углами, на которых указано название состояния. Переходы отмечены стрелками, которые переходят из одного состояния в другое, показывая, как состояния меняются.



Рис. 1. Графические примитивы диаграммы автоматов UML

Основная нотация диаграммы автоматов представлена на рисунке 2.

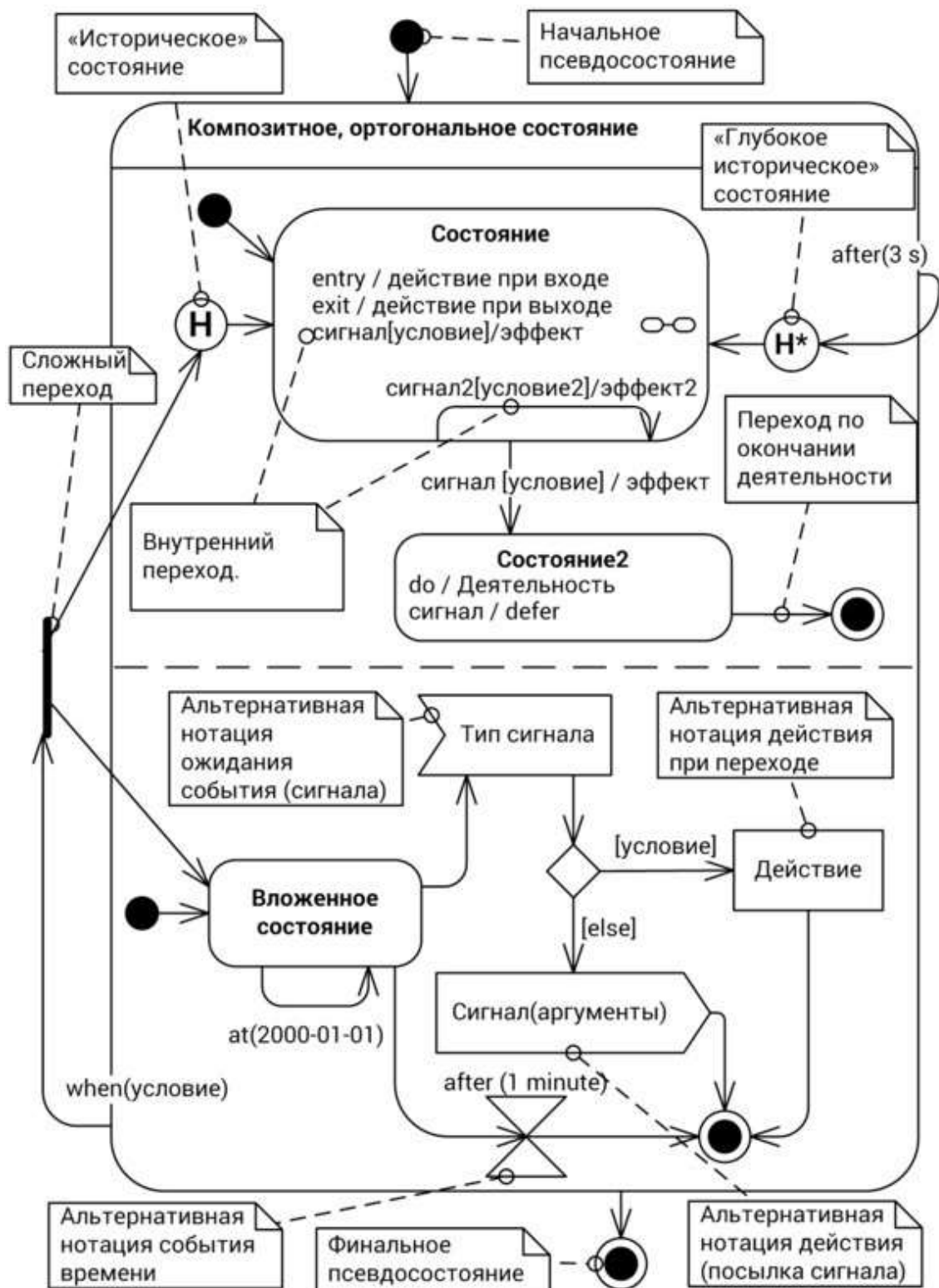


Рис.2. Основная нотация диаграммы состояний

Примеры

На рис. 7.3 показана контекстная диаграмма, моделирующая реакцию системы на выбор пользователем пунктов меню.

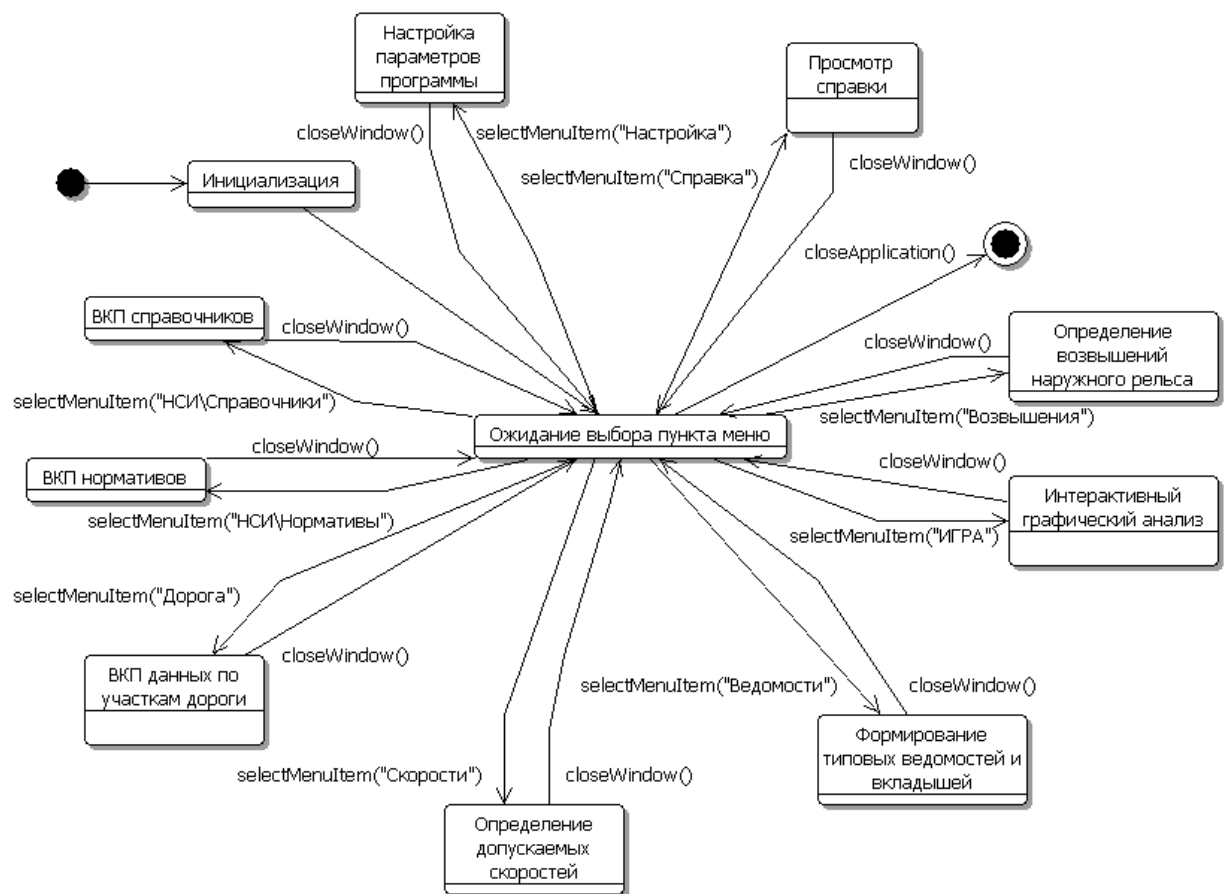


Рис. 3. Контекстная диаграмма автоматов системы определения допускаемых скоростей

На диаграмме приняты следующие условные обозначения:

- ВКП – ввод, корректировка или просмотр данных;
- selectMenuItem() – выбор пункта меню;
- closeWindow() – закрытие окна;
- closeApplication() – закрытие программы.

При запуске программы вначале осуществляется ее инициализация (чтение текущих настроек и отображение основного окна программы). Далее программа автоматически (по нетриггерному переходу) оказывается в состоянии ожидания выбора пункта меню. После выбора пункта меню осуществляется отображение соответствующего диалогового окна на экране, выполнение необходимых действий (как правило, в интерактивном режиме) и закрытие окна с возвратом в исходное состояние.

На следующем рисунке показана диаграмма декомпозиции для составного состояния «ВКП нормативов».

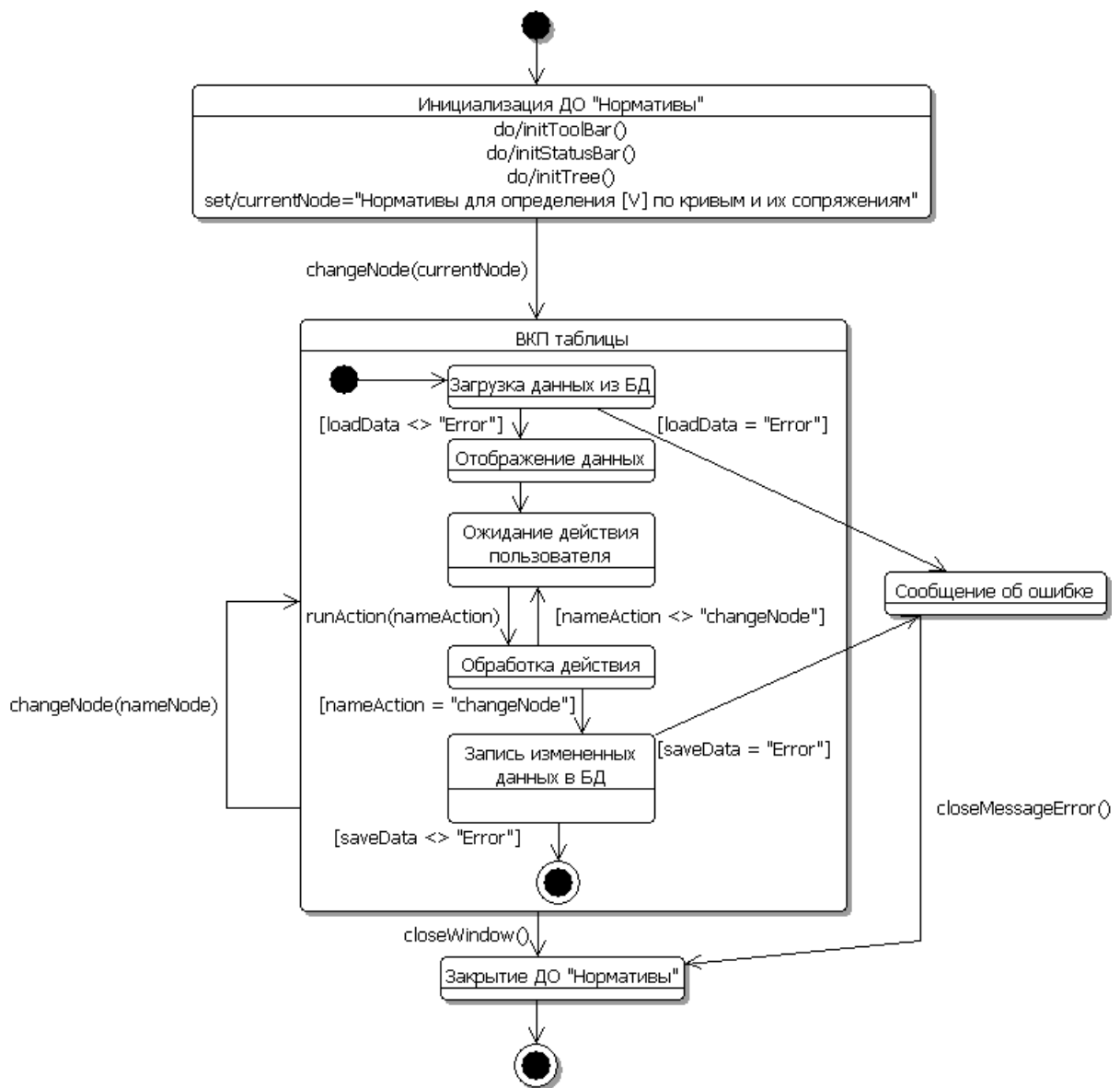


Рис. 4. Диаграмма декомпозиции для состояния «ВКП нормативов»

При выборе пункта меню «Нормативы» на экране отображается соответствующее диалоговое окно. В момент инициализации создаются панель инструментов `initToolBar()`, строка сообщений `initStatusBar()` и дерево с наименованиями таблиц в левой части диалогового окна `initTree()`. Исходным выбранным узлом дерева устанавливаются «Нормативы для определения [V] по кривым и их сопряжениям». При выборе узла дерева `changeNode()`, связанного с таблицей, система выполняет стандартные операции, начиная от загрузки данных, непосредственной обработки действий пользователя `runAction()` с таблицей или ее полями (редактирование значений, вставка новой записи, удаление записи, печать таблицы и т. д.) и заканчивая сохранением в базе всех измененных данных. Если в момент загрузки или записи данных возникает ошибка, то на экране отображается соответствующее сообщение. Закрытие окна «Нормативы» предусмотрено после закрытия сообщения об ошибке `closeMessageError()` или в результате стандартного действия пользователя `closeWindow()`.

Пример состояния "Идентификация пользователя":



Рис.5. Фрагмент диаграммы автоматов для состояния "Идентификация пользователя"

Диаграмма автоматов для режима "Справочники" ИС "Учёт успеваемости студентов":



Рис.6. Диаграмма автоматов для режима "Справочники" ИС "Учёт успеваемости студентов"

Задачи для самостоятельной работы

7.1. При выполнении проекта *Run* руководитель проекта (РП) осуществляет отслеживание текущего статуса *Monitor*. Если выявляется проблема *problem*, то РП устраняет ее *ManageProblem*, и возвращается *succeed* к отслеживанию. Если РП в течение 10 дней не удастся решить проблему *fail*, то происходит завершение проекта *Shutdown* с выполнением остановки *stop*, по завершении которой поведение руководителя проекта завершается.

Добавьте этап работы РП по подготовке *Initiate* проекта. На этом этапе при необходимости *estim* производится оценка *preparePlan* для проектов с бюджетом *budget* более 100К. Для остальных проектов готовится *sketch* набросок. По утверждении *approve* проекта РП переходит к управлению им *Run*.

Покажите на диаграмме, что при управлении проектом РП взаимодействует с другими *Communicate*. Если поступает запрос на собрание *meetingRequest*, то РП его назначает *schedule*. По завершении *done* собрания *RunMeeting*, РП возвращается к взаимодействию.

Добавьте возможность РП приостановить *suspend* работы в любой момент. Если работы приостановлены более года, то РП завершает проект

Shutdown. При возобновлении *resume*, выполнение проекта продолжается с того момента, когда он был приостановлен.

7.2. Моделируется мобильное приложение – электронный словарь, имеющее несколько окон. Схему перехода между окнами будем моделировать с помощью диаграммы схем состояний.

Добавьте на диаграмму состояния *Список слов*, *Перевод*, *Словоформы слова*. Переход между *Список слов* и *Перевод* по событию *Выбрано слово*, обратный переход по событию *Назад*. Переход между *Перевод* и *Словоформы слова* по событию *Словоформы*, обратный переход по событию *Назад*.

В некоторых карточках перевода есть ссылки на другие карточки. При переходе по ссылке мы попадаем в то же состояние *Перевод*, но для другого слова. Добавьте переход в себя из состояния *Перевод*, активируемый событием *Переход по ссылке*.

Возврат после перехода по ссылке по событию *Назад* должен возвращать пользователя в предыдущую карточку. Для этого добавим к переходу по ссылке действие *Добавить в стек*. Добавим переход в себя из состояния *Перевод* по событию *Назад* со сторожевым условием *[стек не пуст]* и действием *Убрать из стека*. В переход из состояния *Перевод* в состояние *Список слов* по событию *Назад* нужно добавить сторожевое условие *[else]*.

Перед запуском приложения нужно загрузить словари. На время загрузки словарей пользователю будет показан экран приветствия. Добавим ортогональное состояние *Инициализация*. В нем выделим два региона. В первом регионе добавим начальное и конечное псевдосостояния. Между ними добавим состояние *Загрузка словарей* с выполняемой при нахождении в состоянии деятельностью *Загрузить словари*. Ожидается переход в конечное состояние по завершении деятельности. Во втором регионе мы добавим переход из начального состояния в конечное по событию времени *after (3 s)*. Это необходимо, чтобы при быстрой загрузке пользователь успел прочитать содержимое экрана приветствия. Из ортогонального состояния *Инициализация* переход по завершению в *Список слов*.

Уточним поведение в окне списка слов. *Список слов* становится составным состоянием. Дальнейшие состояния являются вложенными в *Список слов*. Из начального псевдосостояния происходит переход в состояние *История запросов*. По событию *Ввод* из *Истории запросов* происходит переход в псевдосостояние выбора. Далее при условии *поле ввода пустое* происходит переход (возврат) в состояние *История запросов*. Иначе происходит переход во вложенное составное состояние *Поиск слова*. По событию *Ввод* происходит переход из *Поиск слова* в описанное выше псевдосостояние выбора. В составном состоянии *Поиск слова* непосредственно процедура поиска откладывается с помощью события времени *after (1 s)*. После этого события происходит переход из начального псевдосостояния в состояние *Поиск в словаре*. По завершению происходит переход из *Поиск в словаре* в *Отображение списка*.

Часть 2. Диаграмма деятельности.

Цель занятия: получить навыки построения диаграмм деятельности.

Теоретические сведения к практическому занятию

Диаграмма деятельности — технология, позволяющая описывать логику процедур, бизнес-процессы и потоки работ

Основная нотация диаграммы представлена на рисунке 7

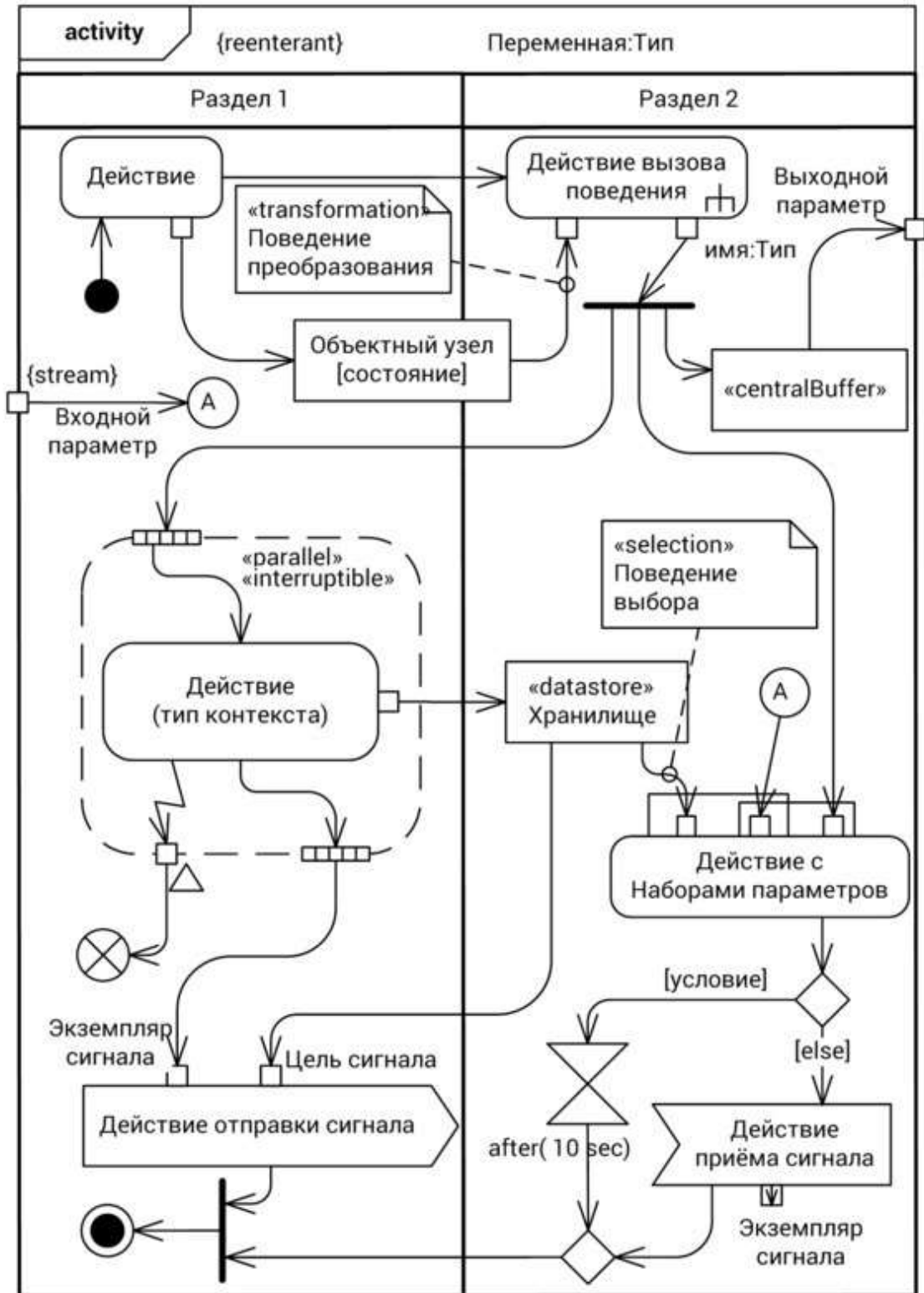


Рис. 7. Основная нотация диаграммы деятельности

Основным отличием диаграмм деятельности от блок-схем является активная поддержка параллельных процессов, что объясняет применение диаграммы деятельности для моделирования потоков работ.

Некоторые из наиболее распространенных компонентов диаграммы активности включают:

Действие: этап действия, на котором пользователи или программное обеспечение выполняют заданную задачу. Действия обозначаются прямоугольниками с закругленными краями.

Узел принятия решения: условная ветвь в потоке, представленная ромбом. Он включает один вход и два или более выходов.

Потоки управления: другое название соединителей, которые показывают поток между шагами на схеме.

Начальный узел: символизирует начало занятия. Начальный узел представлен черным кружком.

Конечный узел: представляет последний этап действия. Конечный узел представлен обведенным черным кружком.

Для моделирования бизнес-процесса можно использовать дорожки – диаграмма с разделением деятельности.

Примеры

На рис. 8 показана диаграмма деятельности для подпотока входа в систему:

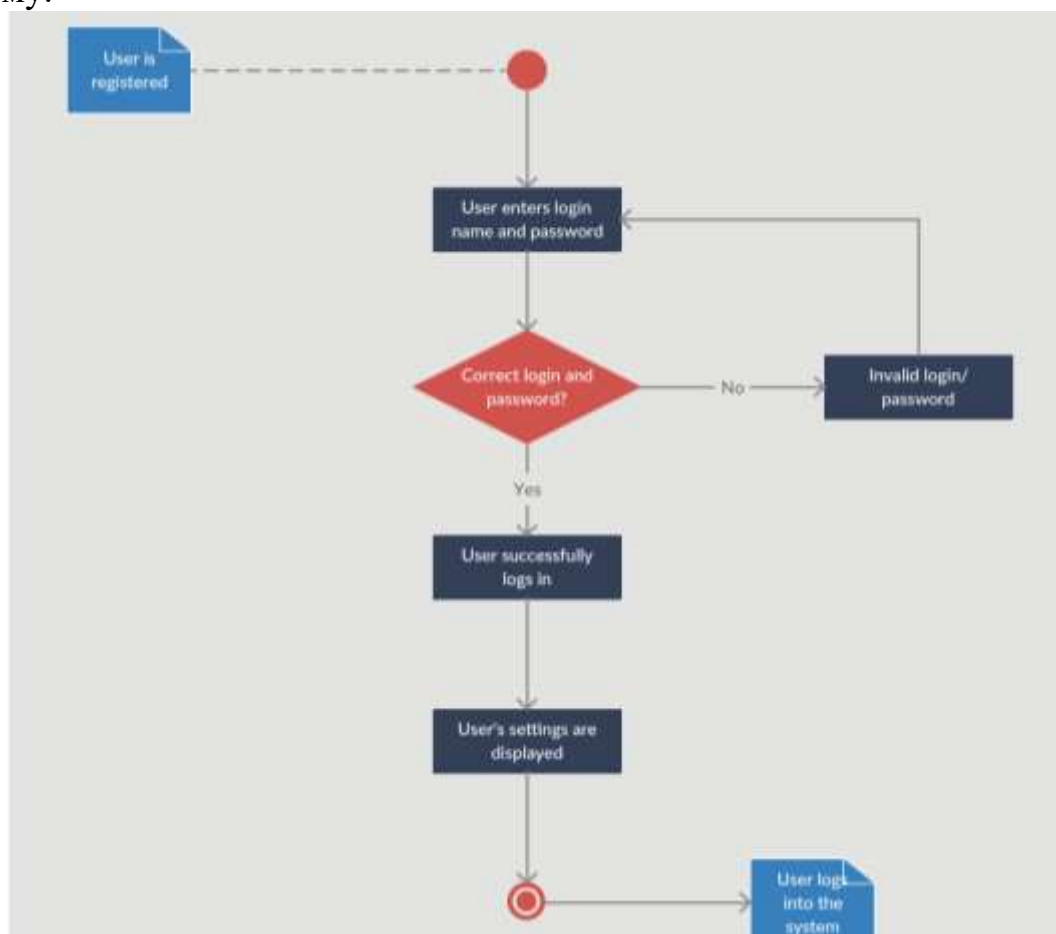


Рис. 8. Диаграмма деятельности для действия Вход в систему

На рис. 9 показана диаграмма деятельности для подпотока «Составить документ».

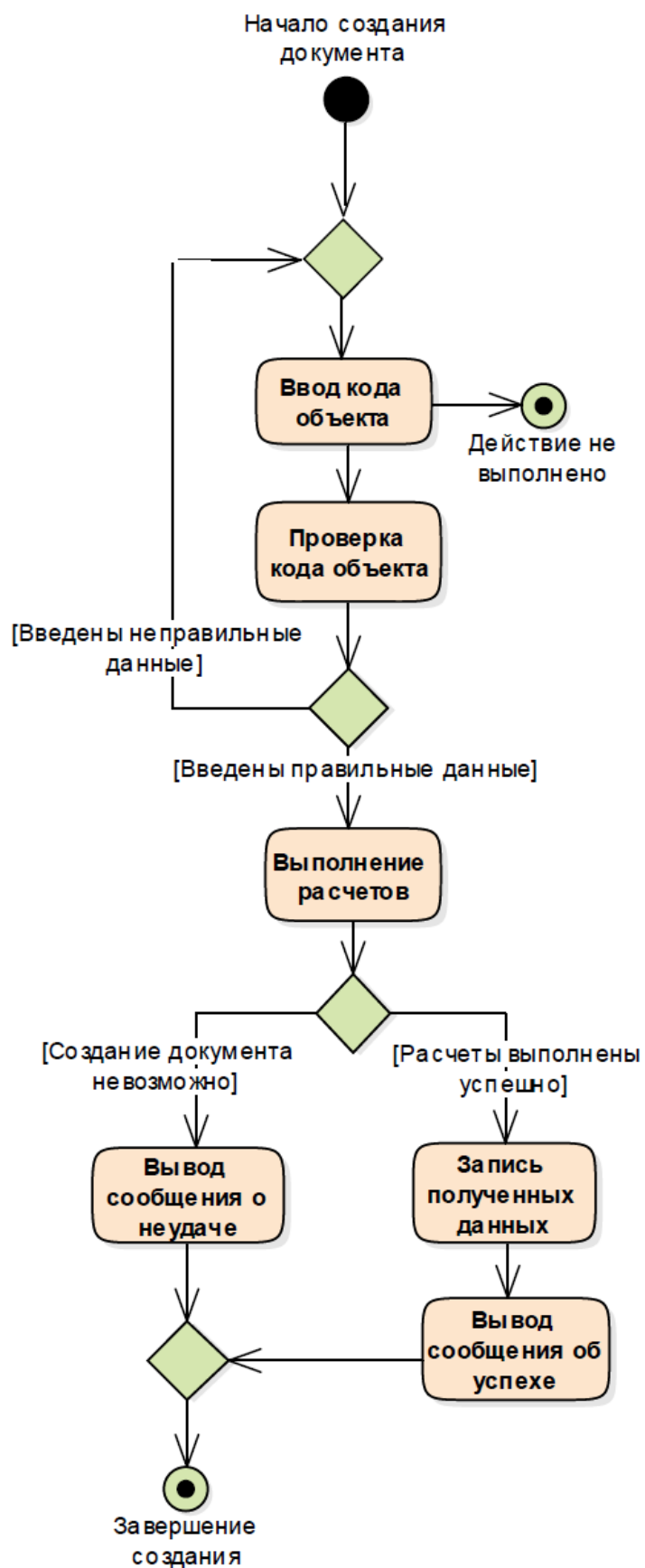


Рис. 9. Диаграмма деятельности для действия «Составить документ»

На рис. 10 показана диаграмма деятельности для системы «Управление школой»:

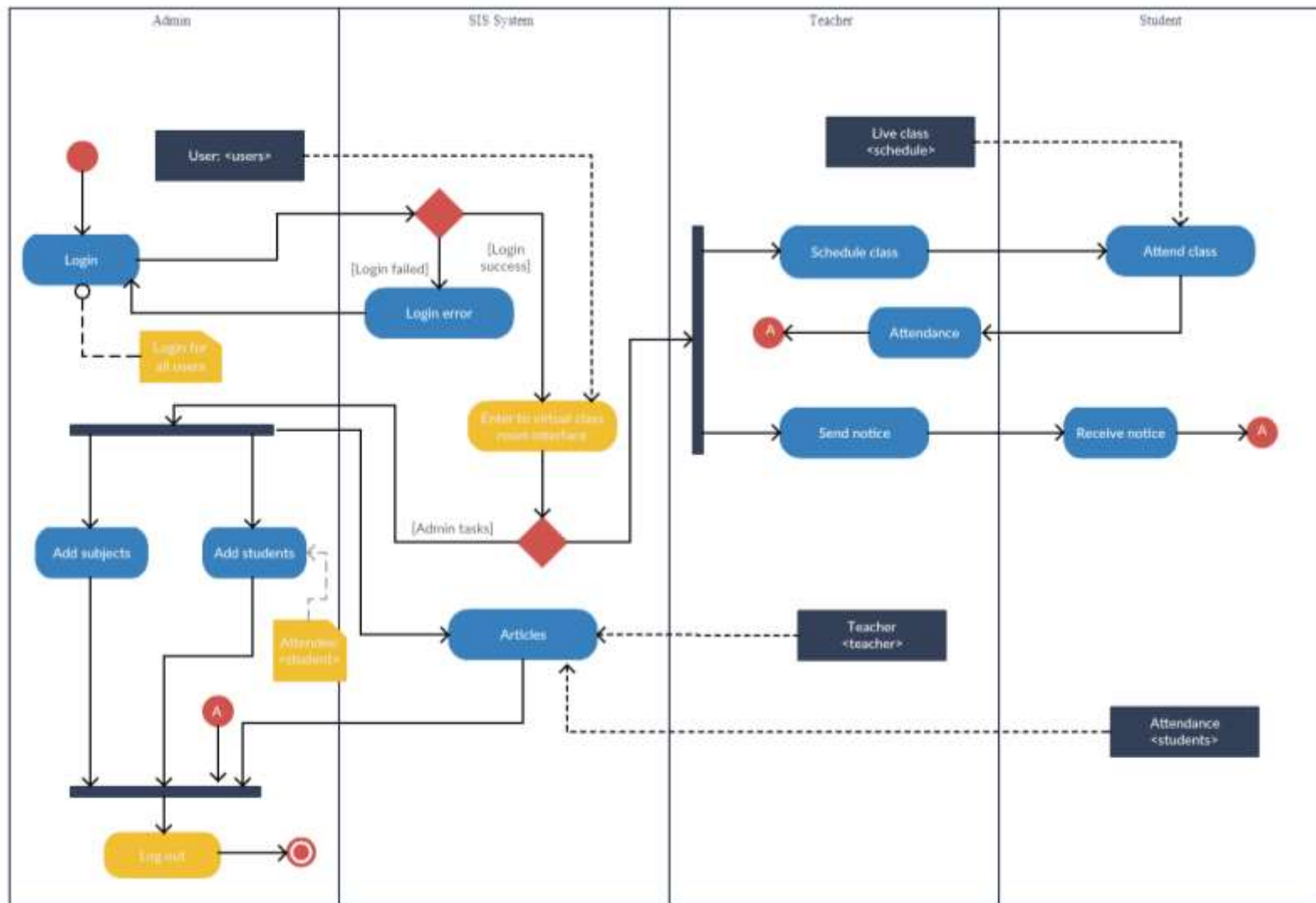


Рис. 10. Диаграмма деятельности для процесса управления школой

На рис. 11 показана диаграмма деятельности для системы управления документами

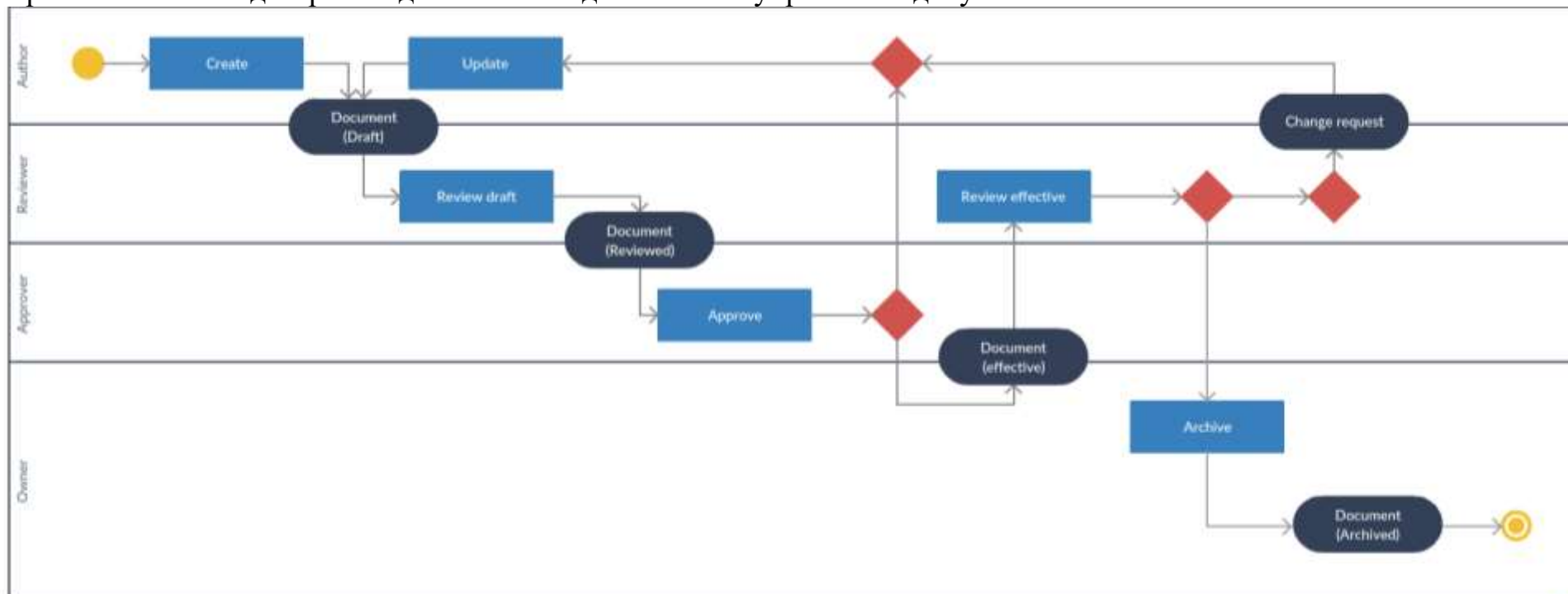


Рис. 11. Диаграмма деятельности для процесса управления документами

Задачи для самостоятельной работы

7.3. Деятельность *ImplementSolution* по реализации автоматизированной системы (АС) начинается с общения *Communication* аналитика *Analyst* с заказчиком. Созданное описание проекта *SoW* используется аналитиком для моделирования *Modeling*. Разработанная в результате модель *Model* сохраняется в репозитории проекта и передается в разработку *Development* и тестирование *Testing*, выполняемые проектной командой *DevTeam*. Результат разработки *Product* передается на тестирование, после этого производится его внедрение *Deployment* интегратором *Implementer*. Отчет о внедрении *Report* является результатом деятельности и деятельность завершается.

а. Покажите, если тестирование обнаружило дефекты в продукте, деятельность возвращается к разработке.

б. В условии задачи не сказано, что описание проекта используется для подготовки к внедрению интегратором. Добавьте данное действие и укажите, что внедрение выполняется только после завершения подготовки.

7.4. Рассмотрим процесс управления продуктовой линейкой *RunSPL*. Сначала заинтересованные стороны *stakeholders* ожидают поступления изменений *WaitChange*. При поступлении изменения *change*, оно передается для оценки *ReviewApprove* комитету *ReviewCommittee*. Результатом оценки является спецификация *spec*, которая передается далее рабочей группе линейки *SPLTeam* для планирования *Plan*. В результате планирования получается проект *project*, который также является результатом процесса.

а. Сохраняйте все спецификации для последующего анализа.

б. Покажите, что если в результате оценки изменение было отклонено *rejected*, то выполнение процесса управления возвращается к ожиданию изменений. Если же изменение было принято *approved*, то процесс переходит к планированию спецификации и ожиданию новых изменений.

7.5. Первым действием деятельности является добавление элемента в заказ *AddItem*. Если заказ сформирован (условие *ready*), то он направляется получателю действием *ShipOrder* и деятельность завершается. В противном случае добавляется еще один элемент заказа.

а. Используя управляющие узлы деятельности, добавьте действие отправки счета *SendInvoice* так, чтобы оно исполнялось после того, как заказ сформирован, независимо от отправки заказа. При этом деятельность не завершается без отправки счета.

б. Предоставьте возможность после внесения элемента, добавить еще элемент в заказ при условии *continue* или завершить выполнение деятельности при условии *cancel*.